

Gestion del proyecto en GitHub - Kenji

Gestión del proyecto en GitHub — guía del líder (Kenji)

Esta guía tiene dos partes: **(A)** el arranque, que haces una sola vez hoy para dejar el repositorio publicado; y **(B)** el gobierno del proyecto durante el semestre — cómo se revisa, se delega y se lleva orden, para no volver a “subir archivos a la mala”.

Se asume que ya tienes la carpeta espejo `Triverde-repo/` armada y que tienes `git` y `gh` (la CLI de GitHub) instalados y con sesión iniciada.

Parte A — Arranque (una vez, hoy)

A.1 Inicializar el repositorio local

Desde la carpeta del espejo:

```
cd C:/Users/Kenji/Proyectos/Triverde-repo
git init -b main
git add .
git commit -m "Primer commit: estructura del repositorio, material de referencia y guía"
```

A.2 Crear el repositorio remoto y subir

Con la CLI de GitHub, en un solo paso (privado, y sube `main`):

```
gh repo create Triverde --private --source=. --remote=origin --push
```

Si prefieres hacerlo a mano: crea el repo vacío en github.com (sin README), y luego `git remote add origin <URL>` y `git push -u origin main`.

A.3 Invitar al equipo

Los invitas por correo desde la web (*Settings* → *Collaborators* → *Add people*), o por CLI:

```
gh repo add-collaborator <usuario-o-correo> --permission push
```

`push` (Write) es el permiso correcto: pueden crear ramas y PRs, pero no borrar el repo ni cambiar su configuración. Repite por cada integrante.

A.4 Proteger `main`

Esto es lo que obliga a que todo pase por PR y revisión. En la web: *Settings* → *Branches* → *Add branch ruleset* (o *Branch protection rule*) para `main`, y activa:

- **Require a pull request before merging** — nadie mete cambios directos a `main`.
- **Require approvals: 1** — cada PR necesita al menos una revisión aprobada.
- **Require branches to be up to date before merging** (opcional pero recomendado).
- Deja *Do not allow bypassing* apagado para ti al principio, por si necesitas arreglar algo urgente; cuando el equipo agarre ritmo, actívalo para todos.

Nota: la protección de ramas es gratis en repos privados con GitHub Free desde 2023, así que no necesitas plan de pago.

Parte B — Gobierno del proyecto durante el semestre

B.1 El flujo, en una línea

Rama por tarea → commits → push → **Pull Request** → **revisión de un compañero** → merge a `main`. Tú no eres el único que revisa: reparte las revisiones para que todos aprendan a leer el trabajo del otro (y para que a ti no se te haga cuello de botella).

B.2 Delegar con Issues

Un **Issue** es una tarea con dueño, discusión y estado. Es la forma ordenada de repartir (mejor que WhatsApp, que se pierde). Por cada tarea pendiente:

1. *Issues* → *New issue*. Título claro en imperativo: “Diagrama de casos de uso — Módulo 4”.
2. En el cuerpo: qué se espera, qué documento de referencia usar (apunta al PDF en `docs/referencia/`) y dónde dejar el resultado (`docs/trabajo-grupal/...`).
3. **Assignees**: la persona responsable. **Labels**: por tipo (`diagrama`, `documento`, `codigo`).
4. Cuando alguien abre el PR que resuelve el Issue, que escriba `Closes #12` en la descripción: al hacer merge, el Issue se cierra solo.

El índice de tareas (`docs/guias/Como trabajamos - indice de tareas.pdf`) ya trae la lista de pendientes lista para convertir en Issues.

B.3 El tablero (Project) — tu kanban del semestre

Para ver todo de un vistazo sin depender de la memoria:

1. En el repo: pestaña **Projects** → **New project** → **Board**.
2. Columnas: **Por hacer** • **En curso** • **En revisión** • **Hecho**.
3. Agrega los Issues al tablero. Cada uno se mueve de columna según avanza. Los PRs también aparecen.

Esto reemplaza el “subir a la mala”: en todo momento se ve quién tiene qué y en qué va.

B.4 El informe de commits (exigencia de la rúbrica)

El profesor pide evidencia de que **todos** commitearon. GitHub te lo da hecho:

- *Insights* → *Contributors*: gráfico de aportes por persona. Ese es el informe.
- **Verifica temprano** (esta semana) que los commits de cada uno aparecen con su nombre. Si alguien sale como “unknown” o con otro nombre, es porque su `git config user.email` **no coincide con un correo verificado en su cuenta de GitHub** (ver la guía del equipo, §1.2). Corregirlo tarde es doloroso: los commits viejos no se reatribuyen fácil. Por eso se drilla al principio.

Regla del ramo: quien no participa en los commits, reprueba. Que nadie deje todo su aporte para el final en un solo commit gigante — commits repartidos en el tiempo cuentan la historia real.

B.5 Ritmo sugerido

- **Inicio de cada sprint:** creas/actualizas los Issues del incremento y los pones en el tablero.
- **Durante:** cada quien trabaja su rama y abre PRs; las revisiones no se acumulan (idealmente se revisan en el día).
- **Cierre:** revisas *Insights* para confirmar aportes parejos y que el tablero quedó en **Hecho**.

B.6 Qué NO versionar

Ya está cubierto por el `.gitignore`, pero para tu tranquilidad: nada de `_privado/`, ni `.env`, ni `node_modules/`, ni la base de datos local. Tu taller personal (`Triverde/` con los `.md` fuente y el pipeline) **no es este repo** y no se sube: aquí va solo lo que el equipo necesita, ensamblado.